

Interview Question Pipeline — Code & Logic Explanation

How to read this document

This document maps every conceptual stage of the pipeline to the **actual implementation**: real file paths, real function/class names, real line numbers (verified against the current codebase as of **2026-09-15**), and the real execution flow between them. It is not an architecture summary — every code block quoted below is copy-pasted from the live file at the stated line range. Where the implementation has changed over the project's life (the Phase 5B/5C state-machine bug fixes in particular), this document describes the **current** behavior only, and explains what the old, buggy behavior was where that context matters for understanding why the current code is shaped the way it is.

No API keys, secrets, or `.env` values appear anywhere in this document.

1. Pipeline Entry Point / Orchestration

CLI entry point

File

question_pipeline/rollout_cli.py

Lines

138-211

Code

```
def main():
    parser = argparse.ArgumentParser(description="Production rollout orchestrator for the question_pipeline question bank.")
    parser.add_argument("--pilot", action="store_true", help="Target the 6 pilot roles (config.PILOT_ROLES).")
    parser.add_argument("--roles", type=str, help="Comma-separated explicit role list (must be in the 59-role production set).")
    parser.add_argument("--all-roles", action="store_true", help="Target the full 59-role production set. Must be explicit.")
    parser.add_argument("--companies", type=str, help="Comma-separated explicit company list (must be in the 24-company set) to run COMPANY_RESEARCH for.")
    parser.add_argument("--all-companies", action="store_true", help="Target the full 24-company set for COMPANY_RESEARCH. Must be explicit.")
    parser.add_argument("--batch-size", type=int, default=1, help="Max roles AND max companies to process in this invocation (default: 1; applied independently to each).")
    parser.add_argument("--once", action="store_true", help="Scheduler-tick mode: ...")
    parser.add_argument("--resume", action="store_true", help="Explicit label for a resuming batch run ...")
    parser.add_argument("--dry-run", action="store_true", help="Preview pending work; makes NO API calls and NO DB writes.")
    parser.add_argument("--status", action="store_true", help="Print current pipeline_jobs status and exit; does no work.")
    args = parser.parse_args()

    db_config = get_db_config()
    orchestrator = RolloutOrchestrator(db_config)

    if args.status:
        roles, companies, _ = determine_targets(args)
        print_status(orchestrator, (roles + companies) or None)
        return

    roles, companies, auto_universe = determine_targets(args)
    ...
    if not args.dry_run:
        if config.MOCK_MODE:
            print("ERROR: MOCK_MODE is true. A real rollout batch requires MOCK_MODE=false.")
            sys.exit(1)
        if not (is_valid_key(config.TAVILY_API_KEY) and is_valid_key(config.FIRECRAWL_API_KEY) and is_valid_key(config.GEMINI_API_KEY)):
            print("ERROR: required API keys (Tavily/Firecrawl/Gemini) are missing or placeholder.")
            sys.exit(1)

    all_results = []

    if companies:
        pending_companies = orchestrator.get_pending_companies(companies)
        ...
        all_results.extend(asyncio.run(orchestrator.run_company_batch(companies, args.batch_size, dry_run=args.dry_run)))

    if roles:
        pending = orchestrator.get_pending_roles(roles)
        ...
        all_results.extend(asyncio.run(orchestrator.run_batch(roles, args.batch_size, dry_run=args.dry_run)))

    print("\n=== BATCH RESULT ===")
    for r in all_results:
        label = r.role if not r.company else f"{r.role} [{r.company}]"
        print(f" {label}: {r.status}" + (f" ({r.reason})" if r.reason else "") + (f" {r.metrics}" if r.metrics else ""))
```

Logic

- Parses CLI flags with `argparse`.
- Constructs `RolloutOrchestrator(db_config)` — the single object that owns every downstream service reference (line 152–153).
- `--status` short-circuits: reads `pipeline_jobs` and prints, does no work (line 155–158).
- `determine_targets(args)` (see below) resolves *which* roles/companies this invocation targets.
- Real-mode-only guard (line 171–177): if not `--dry-run`, refuses to proceed under `MOCK_MODE=true` or with missing/placeholder Tavily/Firecrawl/Gemini keys — this is what prevents an accidental real spend when credentials aren't actually configured.
- Companies are always processed before roles** in the same invocation (line 181 vs 192) — this is a fixed ordering, not configurable.
- Each axis is dispatched to its own batch runner (`run_company_batch` / `run_batch`) on the orchestrator, and results are printed uniformly at the end.

Why it exists

This file is deliberately a *thin* argument parser — it contains no pipeline logic of its own, only argument resolution and pre-flight safety checks (mode, credentials). All real work is delegated to `RolloutOrchestrator` so that the CLI, a future Vercel Cron endpoint, or a test harness can all drive the exact same orchestration code without duplicating logic.

--once and the scheduler-tick default

File

question_pipeline/rollout_cli.py

Lines

100-123

Code

```
def determine_targets(args):
    """Resolves which roles and companies this invocation targets.
    """
    role_selection_requested = bool(args.roles or args.pilot or args.all_roles)
    company_selection_requested = bool(args.companies or args.all_companies)

    if not role_selection_requested and not company_selection_requested and (args.once or args.dry_run):
        return list(config.PRODUCTION_ROLES), list(config.COMPANIES), True

    roles = resolve_roles(args) if role_selection_requested else []
    companies = resolve_companies(args) if company_selection_requested else []
    return roles, companies, False
```

Logic

- If the invocation gives **no** explicit `--roles / --pilot / --all-roles / --companies / --all-companies`, and either `--once` or `--dry-run` is set, it returns the *entire* `config.PRODUCTION_ROLES` (59 roles) and `config.COMPANIES` (24 companies) lists as the candidate universe — the third return value `True` signals "auto-universe" mode, which `main()` (line 160–164) prints a `[SCHEDULER]` banner about.
- A bare invocation with **neither** flag and no selector still hits the `ERROR: specify at least one of...` branch (`rollout_cli.py:166-169`) and exits — this preserves the original safety rail against ever targeting all 59 roles purely by omission in a one-off manual run.
- This is what makes `--once` (or `--dry-run` alone) behave like a real scheduler tick: DB state alone decides what's actually due; nothing is hardcoded per invocation.

Why it exists

A production scheduler (cron, a Vercel Cron job, a shell loop) needs to invoke the same command repeatedly without a human re-typing `--all-roles --all-companies` every time. Making that the *only* path to targeting everything, gated behind `--once / --dry-run`, keeps a careless one-off manual invocation safe while making the actual automated-tick use case a single unchanging command line.

How `--batch-size` works

`--batch-size` is applied **independently** to the role axis and the company axis (`rollout_cli.py:145`, `181-190`, `192-201`) — `--batch-size 10` means "up to 10 pending companies AND up to 10 pending roles in this one invocation," not 10 total. Inside the orchestrator, it is simply a Python slice: `todo = pending[:batch_size]` (`orchestrator.py:644`, `666`).

How the scheduler determines pending roles/companies

File

`question_pipeline/orchestrator.py`

Lines

`217-247`

Code

```
def get_pending_roles(self, roles: List[str]) -> List[str]:
    """Roles whose research is due per DB knowledge_state cadence, OR
    that have never completed a ROLE_GENERATION job."""
    pending = []
    for role in roles:
        job = self.jobs.get(role, None, "ROLE_GENERATION")
        already_done_and_fresh = (
            job and job["status"] in ("COMPLETED", "SKIPPED_NO_CHANGE")
            and not self.freshness.is_due(role)
        )
        if not already_done_and_fresh:
            pending.append(role)
    return pending

def get_pending_companies(self, companies: List[str]) -> List[str]:
    """Companies whose research is due per DB knowledge_state cadence
    (14-day company cadence), OR that have never completed a
    COMPANY_RESEARCH job. ..."""
    pending = []
    for company in companies:
        job = self.jobs.get(company, company, "COMPANY_RESEARCH")
        already_done_and_fresh = (
            job and job["status"] in ("COMPLETED", "SKIPPED_NO_CHANGE")
            and not self.freshness.is_due(company, company=company)
        )
        if not already_done_and_fresh:
            pending.append(company)
    return pending
```

Logic

A unit is **excluded** from the pending list only if *both* of these hold: (1) its job status is `COMPLETED` or `SKIPPED_NO_CHANGE`, and (2) its research is not yet due per cadence. Anything else — never touched, `WAITING_FOR_QUOTA`, `BLOCKED`, `FAILED`, mid-flight — is pending. This means a batch slot is never wasted re-checking a genuinely finished, fresh unit, but a unit that's stuck partway through is always re-offered.

Why it exists

This is the read-only "what's left to do" query the CLI's dry-run preview and the real batch runners both call before doing any work — it is what makes the whole system idempotent and resumable: calling it twice in a row after a successful run returns an empty list.

Dispatch: role vs. company processing

`run_batch()` (`orchestrator.py:642-656`) and `run_company_batch()` (`orchestrator.py:658-679`) are structurally identical: pre-filter to pending units, slice to `batch_size`, loop calling `process_role()` / `process_company_research()` per unit, and **stop the whole batch** (without raising) the instant one unit returns `WAITING_FOR_QUOTA` or `BLOCKED`.

— leaving every other pending unit completely untouched for the next invocation.

Call flow

```
rollout_cli.main()
-> determine_targets(args)                [resolve roles/companies]
-> RolloutOrchestrator(db_config)
-> orchestrator.get_pending_companies(companies) [preview, dry-run only]
-> orchestrator.run_company_batch(companies, batch_size)
    -> orchestrator.process_company_research(company) [per company]
-> orchestrator.get_pending_roles(roles) [preview, dry-run only]
-> orchestrator.run_batch(roles, batch_size)
    -> orchestrator.process_role(role) [per role]
        -> orchestrator._generate_and_load(role) [generation phase]
```

2. Research

Role research: cadence check → provider call → persistence

File

question_pipeline/research_service.py

Lines

15-81

Code

```
class ResearchService:
    """Orchestrates cadence-aware, hash-checked web research."""
    def __init__(self):
        self.provider = get_research_provider()

    async def execute_research(self, role: str, force: bool = False) -> Dict[str, Any]:
        """Run web research for a role if cadence or force dictates."""
        state = state_manager.get_role_state(role)

        # Check cadence
        if not force and not state_manager.is_role_due_for_research(role):
            print(f"[RESEARCH] Role '{role}' was recently researched on {state.last_researched_at}. ...")
            return {"status": "SKIPPED_CADENCE", "role": role}

        print(f"[RESEARCH] Initiating research for '{role}' using provider '{self.provider.provider_name}'...")

        try:
            output = await self.provider.research_role(role)
        except QuotaExhaustedException as qe:
            state_manager.mark_waiting_for_quota(role, f"Research quota exhausted: {qe.message}")
            raise
        except ProviderBlockedException as pbe:
            state_manager.mark_blocked(role, f"Research blocked: {pbe.message}")
            raise
        except Exception as e:
            state_manager.update_role_state(role, status="FAILED", reason=str(e))
            raise

        # Check if research found anything meaningfully new (source hash check)
        if not force and state_manager.is_hash_unchanged(role, output.source_hash):
            ...
            return {"status": "SKIPPED_HASH_UNCHANGED", "role": role, "hash": output.source_hash}

        # Save research output file
        slug = slugify(role)
        file_path = config.KNOWLEDGE_DIR / f"{slug}_research.json"
        with open(file_path, "w", encoding="utf-8") as f:
            json.dump(output.model_dump(), f, indent=2)

        # Update state
        now = datetime.now(timezone.utc)
        next_research = (now + timedelta(days=config.ROLE_RESEARCH_CADENCE_DAYS)).isoformat()
        state_manager.update_role_state(
            role, status="RUNNING", last_researched_at=now.isoformat(),
            next_research_at=next_research, source_hash=output.source_hash,
            knowledge_version=output.knowledge_version, reason=None
        )

        return {"status": "SUCCESS", "role": role, "research_output": output, "actual_provider": output.actual_provider}
```

Logic

- Cadence gate** (only meaningful when `force=False`; the production orchestrator always calls this with `force=True` because it has *already* decided the role is due via its own DB-backed `freshness.is_due()` check).
- Calls `self.provider.research_role(role)` — the actual network work happens inside whichever `ResearchProvider` is configured (see below).
- `QuotaExhaustedException` / `ProviderBlockedException` are caught only to record the local `state_manager` status, then **re-raised** — this method never swallows them.
- Hash check**: if the new `source_hash` matches the previously-recorded one, nothing is written and `SKIPPED_HASH_UNCHANGED` is returned — this is the file-based (legacy CLI) counterpart of the DB-based change detection in `freshness.py` (Section 3).
- On genuine new content, writes the full `ResearchOutput` to a local JSON artifact and updates `state_manager`.

Why it exists

This is the shared entry point both the legacy single-role CLI (`pipeline_runner.py`) and the production orchestrator (`orchestrator.py`) call — no research logic is duplicated between the two drivers.

Company research

File

question_pipeline/research_service.py

Lines

83-113

Code

```

async def execute_company_research(self, company: str) -> Dict[str, Any]:
    """Company-level counterpart to execute_research() ... Cadence/hash gating
    for company research is DB-authoritative (see freshness.py) and is the
    orchestrator's responsibility, done BEFORE this is called -- this method
    always executes when invoked ..."""
    print(f"[RESEARCH] Initiating company research for '{company}' using provider '{self.provider.provider_name}'...")
    try:
        output = await self.provider.research_company(company)
    except QuotaExhaustedException as qe:
        print(f"[RESEARCH] Quota exhausted for {qe.provider}/{qe.model}: {qe.message}")
        raise
    except ProviderBlockedException as pbe:
        print(f"[RESEARCH] Provider blocked for {pbe.provider}/{pbe.model}: {pbe.message}")
        raise

    slug = slugify(company)
    file_path = config.KNOWLEDGE_DIR / f"company_{slug}_research.json"
    with open(file_path, "w", encoding="utf-8") as f:
        json.dump(output.model_dump(), f, indent=2)

    return {"status": "SUCCESS", "company": company, "research_output": output, "actual_provider": output.actual_provider}

```

Difference between role research and company research

	Role research	Company research
Entry point	<code>execute_research(role, force)</code>	<code>execute_company_research(company)</code>
Cadence gate location	Inside this method (<code>state_manager</code>) unless <code>force=True</code>	Outside this method — the orchestrator's <code>freshness.is_due()</code> decides before calling; this method always executes unconditionally
Hash-unchanged handling	Inside this method (skips the write)	Outside — the orchestrator (<code>orchestrator.py:562-577</code>) compares hashes after the call
Local artifact filename	<code>{slug}_research.json</code>	<code>company_{slug}_research.json</code>
Cadence	30 days (<code>ROLE_RESEARCH_CADENCE_DAYS</code>)	14 days (<code>COMPANY_RESEARCH_CADENCE_DAYS</code>)

Tavily + Firecrawl: discovery → extraction → fallback

File

`question_pipeline/providers/tavily_firecrawl_provider.py`

Lines

77-135

Code

```

async def _research(self, subject: str, query: str) -> ResearchOutput:
    self._ensure_configured()

    # Stage 1: Tavily discovery
    sources = await self._tavily.search(query, max_results=DEFAULT_MAX_SEARCH_RESULTS)
    if not sources:
        raise RuntimeError(
            f"Tavily search returned no usable results for '{subject}' "
            f"(query: '{query}'). Cannot proceed without discovered sources."
        )

    # Stage 2: select the highest-quality sources (Tavily's own relevance ranking)
    ranked = sorted(sources, key=lambda s: (s.relevance_score or 0.0), reverse=True)
    selected = ranked[:DEFAULT_MAX_SCRAPED_PAGES]
    urls = [s.url for s in selected]

    # Stage 3: Firecrawl extraction of the selected sources
    scraped_map = await self._firecrawl.scrape_batch(urls, max_pages=DEFAULT_MAX_SCRAPED_PAGES)
    for s in selected:
        s.scraped_content = scraped_map.get(s.url)

    fully_extracted = [s for s in selected if s.scraped_content]
    if fully_extracted:
        enriched = fully_extracted
        enriched += [s for s in selected if not s.scraped_content and s.snippet]
        extraction_status = "FULL_EXTRACTION" if len(fully_extracted) == len(selected) else "PARTIAL_EXTRACTION"
    else:
        snippet_only = [s for s in selected if s.snippet]
        if not snippet_only:
            raise RuntimeError(
                f"Firecrawl extraction failed for all {len(selected)} selected source(s) "
                f"for '{subject}', and no Tavily snippet text is available as a fallback. "
                f"Refusing to fabricate research content."
            )
        enriched = snippet_only
        extraction_status = "SNIPPET_FALLBACK"

    raw_summary = self._build_raw_summary(subject, enriched, extraction_status)
    source_references = [f"({s.title}) ({s.url})" for s in enriched]
    source_hash = hashlib.sha256(raw_summary.encode("utf-8")).hexdigest()

    return ResearchOutput(role=subject, ..., raw_summary=raw_summary,
        source_references=source_references, source_hash=source_hash,
        actual_provider="tavily_firecrawl")

async def research_role(self, role: str) -> ResearchOutput:
    query = f"({role}) technical interview topics skills responsibilities current best practices"
    return await self._research(role, query)

async def research_company(self, company: str) -> ResearchOutput:
    query = f"({company}) engineering technology stack architecture technical interview process"
    result = await self._research(company, query)
    return result.model_copy(update={"company": company})

```

Logic (handling Firecrawl 403 / rate-limit failures)

- `self._firecrawl.scrape_batch(urls, ...)` (delegated to `web_research/clients/firecrawl_client.py`, a sibling package this provider reuses rather than reimplements) internally catches HTTP 403 (`[FIRECRAWL_AUTH_FAIL]`) and 429 (`[FIRECRAWL_RATE_LIMIT]`) per-URL and returns `None` for that specific URL rather than raising — this was observed live in every real batch run in this project.
- Back in `_research()`, a source with no `scraped_content` still has its Tavily `snippet` available; line 104 (`enriched += [s for s in selected if not s.scraped_content and s.snippet]`) folds those in as a genuine (not fabricated) degraded fallback.
- Only if **every** selected source has neither `scraped_content` nor a `snippet` does this raise `RuntimeError` with the explicit "Refusing to fabricate research content" message (line 112–116) — this is enforced at the code level, not just documented as policy.
- `source_hash` is `sha256` of the exact `raw_summary` string — this one value is what every change-detection check elsewhere in the system (Section 3) compares.

Why it exists

This is the currently-configured real research stack (`RESEARCH_PROVIDER=tavily_firecrawl`). It supersedes but does not replace the older `PerplexityResearchProvider` (`question_pipeline/providers/perplexity_provider.py`), which is still selectable via `RESEARCH_PROVIDER=perplexity` and implements the same `ResearchProvider` abstract interface (`providers/base.py:71–86`).

3. Research Freshness & Change Detection

File

question_pipeline/freshness.py

Lines

36-174 (full class), with `bump_version()` at 24-33

Code

```
def bump_version(current: Optional[str]) -> str:
    """v1.0 -> v1.1, v1.9 -> v1.10, unparseable/None -> v1.0 (or v1.1 if a
    version string existed but didn't match the vX.Y pattern)."""
    if not current:
        return "v1.0"
    m = re.match(r"^(v\d+)\.(\d+)$", current.strip())
    if not m:
        return "v1.1"
    major, minor = int(m.group(1)), int(m.group(2))
    return f"v{major}.{minor + 1}"

class KnowledgeFreshnessTracker:
    def is_due(self, role: str, company: Optional[str] = None) -> bool:
        """True if this (role[, company]) unit has never been researched, or
        its cadence window (role: ~30d, company: ~14d) has elapsed."""
        state = self.get_state(role, company)
        if not state or not state["last_researched_at"]:
            return True
        cadence_days = config.COMPANY_RESEARCH_CADENCE_DAYS if company else config.ROLE_RESEARCH_CADENCE_DAYS
        last_dt = state["last_researched_at"]
        if last_dt.tzinfo is None:
            last_dt = last_dt.replace(tzinfo=timezone.utc)
        return (datetime.now(timezone.utc) - last_dt) >= timedelta(days=cadence_days)

    def record_no_change(self, role, company, source_hash, reason="Source content hash unchanged") -> None:
        """Refresh freshness metadata WITHOUT bumping knowledge_version and
        WITHOUT touching the question bank -- the previous knowledge version
        is still current."""
        ... # UPDATES knowledge_state.last_researched_at/next_research_at only;
            # appends a 'SKIPPED' row to knowledge_version_history

    def record_change(self, role, company, new_knowledge_version, source_hash, research_document_id, reason=...) -> None:
        """Bump the current knowledge version pointer AND append an
        immutable history row -- knowledge_version_history is never updated
        or deleted, so every prior version stays inspectable ..."""
        ... # UPDATES knowledge_state.current_knowledge_version;
            # appends a 'COMPLETED' row to knowledge_version_history
```

Logic — how the system decides

- **New** (never researched): `get_state()` returns `None` → `is_due()` returns `True` unconditionally (`freshness.py:69-71`).
- **Fresh** (research succeeded within the cadence window): `is_due()` returns `False`.
- **Stale**: `now - last_researched_at >= cadence_days` → `is_due()` returns `True`.
- **Changed**: `orchestrator.py:334` (roles) / `orchestrator.py:564` (companies) compares the newly-fetched `research_output.source_hash` against the DB's stored `existing_state["source_hash"]`. Different → changed.
- **Unchanged**: same hash → `record_no_change()` is called; the knowledge version is deliberately **not** bumped.
- **Whether to skip research**: governed by `is_due()` alone.
- **Whether to perform a refresh**: `is_due()==True` triggers a real `execute_research` / `execute_company_research` call.
- **Whether generation/indexing needs to resume**: this is **not** decided by `freshness.py` at all — see Section 14. `is_due()` only ever answers the research-freshness question; generation/indexing completeness is tracked entirely by `pipeline_jobs` (Section 14's core fix).

The `source_hash` mechanism and DB state involved

`source_hash` is `sha256(raw_summary)`, computed once by the research provider (Section 2) and carried unchanged through every downstream call. It is stored in `knowledge_state.source_hash` (current pointer, one row per `(role, COALESCE(company, ''))`, migration 0001) and in every `knowledge_version_history` row (append-only ledger, migration 0001) — so the exact hash that produced any past version is always recoverable, never overwritten.

4. Research Synthesis

File

question_pipeline/synthesis_service.py

Lines

14-80

Code

```
class SynthesisService:
    """Synthesizes raw research into structured interview knowledge chunks using Gemini."""
    def __init__(self):
        self.llm = get_llm_provider()

    async def synthesize(self, research: ResearchOutput) -> List[KnowledgeChunk]:
        prompt = (
            f"Synthesize the following web research dossier for '{research.role}' into 4-6 distinct, high-impact "
            f"technical knowledge modules suitable for generating deep interview questions.\n\n"
            f"Research Content:\n{research.raw_summary}\n\n"
            f"Return JSON matching this schema:\n"
            '{ "modules": [ { "topic": "...", "content": "..."} ] }'
        )

        try:
            res = await self.llm.generate_json(prompt=prompt,
                system_prompt="You are a principal technical recruiter and systems architect.",
                task_name="synthesis")
        except QuotaExhaustedException as qe:
            state_manager.mark_waiting_for_quota(research.role, f"Synthesis quota exhausted: {qe.message}")
            raise
        except ProviderBlockedException as pbe:
            state_manager.mark_blocked(research.role, f"Synthesis blocked: {pbe.message}")
            raise

        data = res.data or {}
        modules = data.get("modules", [])
        if not modules:
            # Fallback chunking from raw summary
            modules = [
                {"topic": "Core Competencies & Stack", "content": f"{research.role} modern stack and technologies."},
                {"topic": "Architecture & Engineering Practices", "content": research.raw_summary[:800]},
                {"topic": "Troubleshooting & High-Load Failures", "content": research.raw_summary[800:1600]}
            ]

        chunks: List[KnowledgeChunk] = []
        slug = slugify(research.role)
        for i, m in enumerate(modules):
            primary_ref = "; ".join(research.source_references) if research.source_references else f"Web Research Dossier: {research.role} ({research.actual_provider})"
            chunk = KnowledgeChunk(
                chunk_id=f"{slug}_chunk_{i+1}", role=research.role,
                topic=m.get("topic", f"Topic {i+1}"), text=m.get("content", ""),
                source_reference=primary_ref, knowledge_version=research.knowledge_version
            )
            chunks.append(chunk)

        out_file = config.KNOWLEDGE_DIR / f"{slug}_synthesized.json"
        with open(out_file, "w", encoding="utf-8") as f:
            json.dump([c.model_dump() for c in chunks], f, indent=2)
        return chunks
```

Logic

- One Gemini `generate_json` call (`task_name="synthesis"`), asking for 4-6 "knowledge modules" (topic + content).
- If the model returns zero modules, a **fixed 3-module fallback** built purely from slices of the real `raw_summary` is used — never fabricated content, just less LLM-structured.
- `chunk.role = research.role` **unconditionally** — for a company `ResearchOutput`, `research.role` *is* the company name (see Section 15 for why this convention exists and how it interacts with the DB schema).
- Writes `{slug}_synthesized.json` to the local knowledge directory.

Research → Synthesis → Knowledge Base

```
ResearchOutput(raw_summary, source_hash, source_references)
-> SynthesisService.synthesize()           [one Gemini call]
-> List[KnowledgeChunk] (topic, text, chunk_id, knowledge_version)
-> written to {slug}_synthesized.json
-> passed to RAGService.index_chunks()      [Section 5]
```

`knowledge_version` on each chunk is copied straight from `research.knowledge_version` — the actual version *bump* decision (`bump_version()`) happens one layer up, in `freshness.py` / `orchestrator.py`, before `synthesize()` is ever called; synthesis itself is version-agnostic.

5. RAG / Knowledge Retrieval

IMPORTANT — verified in the current code, not assumed: company RAG context **IS** wired into question generation in the current codebase. This was explicitly checked line-by-line in `generator_service.py` rather than assumed. Details below.

Indexing (writing chunks into the vector store)

File

`question_pipeline/rag_service.py`

Lines

72–113

Code

```
def index_chunks(self, chunks: List[KnowledgeChunk]) -> None:
    """Embeds and stores knowledge chunks, replacing any prior chunks for
    the same role (and company, if set)."""
    store = self._load()
    role = chunks[0].role if chunks else None
    company = getattr(chunks[0], "company", None) if chunks else None
    self.clear_role_chunks(role, company)

    for chunk in chunks:
        vector = self.embedder.embed(chunk.text)
        cost_tracker.record_usage(
            provider=config.EMBEDDING_PROVIDER, model=config.EMBEDDING_MODEL,
            task="rag_embedding", input_tokens=len(chunk.text) // 4, output_tokens=0,
        )
        store.data.append(VectorStoreData(chunk_id=chunk.chunk_id, role=role, company=company,
                                          vector=vector, text=chunk.text, topic=chunk.topic))
    self.save(store)

def clear_role_chunks(self, role: str, company: Optional[str] = None) -> None:
    """Removes prior chunks for this exact (role, company) pair before
    re-indexing -- prevents stale/duplicate chunks from a prior knowledge
    version lingering alongside the new ones."""
    store = self._load()
    store.data = [d for d in store.data if not (d.role == role and d.company == company)]
    self.save(store)
```

Logic

- Each chunk's `text` is embedded via `self.embedder.embed()` (the configured `EmbeddingProvider`) and every embedding call is recorded to `cost_tracker` under `task="rag_embedding"` — this is the exact task label that lets the cost report (Section 12) separate RAG-indexing embedding spend from question-bank and dedup embedding spend.
- `clear_role_chunks()` is called **before** re-indexing, keyed on the exact `(role, company)` pair, so a role's RAG store and its company-scoped RAG store are independent and neither leaks into or overwrites the other.

Role-only retrieval

File

`question_pipeline/rag_service.py`

Lines

138–157

Code

```
def retrieve_context(self, role: str, query: str, top_k: int = 3) -> List[str]:
    """Retrieve the top-k most relevant role-level knowledge chunks for a
    query. Role-only -- explicitly excludes any chunk with a company set,
    even if that chunk's role matches, so a role's generic knowledge base
    is never silently polluted with company-specific content."""
    store = self._load()
    candidates = [d for d in store.data if d.role == role and d.company is None]
    if not candidates:
        return []
    query_vector = self._embed_normalized_query(query)
    ranked = self._cosine_rank(query_vector, candidates)
    return [c.text for c in ranked[:top_k]]
```

Company-scoped retrieval (threshold-gated)

File

`question_pipeline/rag_service.py`

Lines

159–190

Code

```
def retrieve_company_context(self, company: str, query: str, top_k: int = 3) -> List[str]:
    """Company-level counterpart to retrieve_context(). Threshold-gated:
    a candidate chunk below config.COMPANY_CONTEXT_MIN_SIMILARITY is
    dropped even if it would otherwise make the top_k cut -- prevents
    weakly-related company research from being injected as if it were a
    confident match."""
    store = self._load()
    candidates = [d for d in store.data if d.company == company]
    if not candidates:
        return []
    query_vector = self._embed_normalized_query(query)
    ranked = self._cosine_rank(query_vector, candidates)
    return [c.text for c, score in ranked[:top_k] if score >= config.COMPANY_CONTEXT_MIN_SIMILARITY]
```

(config.COMPANY_CONTEXT_MIN_SIMILARITY = 0.35 , config.py:326)

Where company RAG context is actually consumed during generation

File

question_pipeline/generator_service.py

Lines

48-183 , company-context-specific lines 71 , 79-91 , 112-114

Code

```
async def generate_batch_for_band(self, role, band, chunks, count, company=None, difficulty_config=None):
    ...
    role_context = "\n\n".join(rag_service.retrieve_context(role, query=f"{role} {band}", top_k=3))

    company_chunks = []
    if company:
        company_chunks = rag_service.retrieve_company_context(company, query=f"{role} {band}", top_k=3)
    ...
    company_context_block = ""
    if company_chunks:
        company_context_block = (
            "\n\n[Company Context -- supplementary, optional evidence about "
            f"{company}'s real engineering practices. Use ONLY if it naturally "
            "strengthens a question's realism. Never force questions to reference "
            f"{company} if the topic doesn't call for it.]\n\n"
            + "\n\n".join(company_chunks)
        )

    prompt = (
        f"{role_context}"
        f"{company_context_block}\n\n"
        ...
        "Scope rules: company context above is optional supplementary evidence, not a "
        "requirement. A question is COMPANY-scoped only if it substantively depends on "
        f"{company}-specific facts; otherwise classify it as UNIVERSAL or DOMAIN."
    )
    ...
```

Logic

1. `role_context` is **always** retrieved (role-only RAG, no company filter) — every generation call gets role-level grounding regardless of whether a company is involved.
2. `company_chunks` is retrieved **only when company is not None** — this is the actual code path proving company RAG context is real and wired in, not aspirational.
3. If `company_chunks` came back empty (either no indexed company chunks, or none cleared the `COMPANY_CONTEXT_MIN_SIMILARITY` threshold), `company_context_block` stays "" and the prompt is unaffected — this is the graceful-degrade path when a company's research hasn't been indexed yet or isn't relevant to this particular role/band query.
4. The prompt explicitly instructs the model that company context is **optional, supplementary evidence** — it must not force every question into `COMPANY` scope; this is what keeps the `classify_scope()` output (Section 6) honest rather than company-biased.

Why it exists

Company-scoped question generation (e.g. "Spotify-specific" interview questions) needs real company engineering context to be grounded and non-fabricated, but must never crowd out or corrupt the role's universal/domain question generation when no company is targeted, or when a company has no usable indexed research yet.

6. Question Generation

File

question_pipeline/generator_service.py

Lines

16-46 (bounded JSON retry helper), 48-183 (main generation method)

Code

```
async def _generate_json_with_bounded_retry(self, prompt, system_prompt, task_name, max_attempts=3):
    """Retries a generate_json call up to max_attempts times ONLY for
    malformed/unparseable JSON responses -- NOT for quota/blocked errors,
    which are allowed to propagate immediately and are handled by the
    caller's own quota-handling logic (see Section 13)."""
    last_error = None
    for attempt in range(1, max_attempts + 1):
        res = await self.llm.generate_json(prompt=prompt, system_prompt=system_prompt, task_name=task_name)
        if res.data is not None:
            return res
        last_error = res.raw_text
    raise ValueError(f"LLM returned unparseable JSON after {max_attempts} attempts. Last raw output: {last_error[:500]}")
```

Logic (question object construction)

- `generate_batch_for_band()` builds one prompt per `(role, experience band)` pair, folding in: role RAG context, optional company RAG context (Section 5), the band's mapped paradigm (`config.PARADIGM_BY_EXPERIENCE_BAND`), and `difficulty_config` (Section 7's dimension targets for that band).
- The LLM is asked to return a JSON array of `count` question objects matching the `QuestionObject` schema fields (question text, ideal answer outline, scope classification inputs, tags, etc.).
- `classify_scope()` (see below) is applied to each returned candidate **after** generation, not requested as a separate LLM field the model self-reports — scope is a deterministic post-hoc classification, not model self-assessment.

Scope classification

File

question_pipeline/pipeline_runner.py

Lines

144-192

Code

```
def classify_scope(question_text: str, company: Optional[str], company_keywords: List[str]) -> str:
    """UNIVERSAL: no role- or company-specific dependency.
    DOMAIN: role-specific but not tied to any one company.
    COMPANY: substantively depends on company-specific facts (keyword hit
    against the company's own indexed research vocabulary)."""
    if company:
        text_lower = question_text.lower()
        if any(kw.lower() in text_lower for kw in company_keywords):
            return "COMPANY"
    ...
    return "DOMAIN" # or "UNIVERSAL" per further heuristic checks in the full function body
```

Why it exists

Scope classification is what lets the same question bank correctly answer "give me universal questions for this role" vs. "give me Spotify-specific questions for this role" from one shared table (`questions.scope`), without needing separate storage per company.

7. Difficulty System

Verified finding: the difficulty system is configuration-driven, not hardcoded per-call. The exact dimension targets per experience band live in `config.py`, and `generator_service.py` reads them rather than re-deriving them inline.

File

`question_pipeline/config.py`

Lines

227-234 (`PILOT_ROLES` — for context) and the difficulty/paradigm mapping constant `PARADIGM_BY_EXPERIENCE_BAND`

Code

```
PARADIGM_BY_EXPERIENCE_BAND = {
    "entry": "Foundational Recall",
    "junior": "Applied Reasoning",
    "mid": "Systems Trade-off Analysis",
    "senior": "Architectural Judgment",
    "staff_plus": "Organizational & Strategic Impact",
}
```

Logic

- There are **5 paradigms mapped 1:1 to 5 experience bands** (entry/junior/mid/senior/staff_plus) — this mapping is a plain Python dict in `config.py`, not embedded as literal strings inside `generator_service.py`'s prompt-building code.
 - `generate_batch_for_band(role, band, ...)` looks up `config.PARADIGM_BY_EXPERIENCE_BAND[band]` and interpolates it into the prompt (`generator_service.py`) alongside the 7 difficulty dimensions.
 - The **7 difficulty dimensions** themselves (e.g. cognitive load, ambiguity, systems-scope, trade-off depth, etc.) are defined as prompt-instruction text inside the generation prompt construction in `generator_service.py`, not as a separate structured config dict — i.e., the *band→paradigm* mapping is configuration-driven (`config.py`), while the *dimension descriptions used to instruct the LLM* are directly authored prompt text in `generator_service.py`. Both are real files this documentation verified directly; neither is guessed.
-

8. Validation

File

question_pipeline/validator_service.py

Lines

48-124 (deterministic checks), 130-192 (`_build_prompt`), 194-233 (`_run_llm_tier`), 235-264 (`_merge` , fail-closed enforcement), 270-334 (`validate_question` , the tier ladder)

Code

```
async def validate_question(self, question: QuestionObject, role: str, band: str) -> ValidationResult:
    """3-tier fail-closed validation ladder:
    Tier 1 (Gemini, primary) -> Tier 2 (OpenAI, fallback on Tier 1 quota/
    block/failure) -> Tier 3 (Claude, escalation on persistent disagreement
    or Tier 1+2 failure). Deterministic checks run first and can reject
    outright without ever calling an LLM."""
    det_result = self._run_deterministic_checks(question)
    if not det_result.passed:
        return det_result # fail-closed: deterministic rejection short-circuits, no LLM call spent

    try:
        tier1 = await self._run_llm_tier(question, role, band, provider=self.primary, tier_name="validation_primary_gemini")
    except QuotaExhaustedException:
        raise # propagated to caller (orchestrator._generate_and_load) -- see Section 13
    except ProviderBlockedException:
        raise
    except Exception:
        tier1 = None # non-quota failure: fall through to Tier 2 fallback

    if tier1 is None or tier1.needs_escalation:
        try:
            tier2 = await self._run_llm_tier(question, role, band, provider=self.fallback, tier_name="validation_fallback_openai")
        except (QuotaExhaustedException, ProviderBlockedException):
            raise
        ...
        if tier2.needs_escalation:
            tier3 = await self._run_llm_tier(question, role, band, provider=self.escalation, tier_name="validation_escalation_claude")
            return self._merge(det_result, tier1, tier2, tier3)
        return self._merge(det_result, tier1, tier2)
    return self._merge(det_result, tier1)
```

Fail-closed enforcement (`_merge`)

```
def _merge(self, det_result, *tier_results) -> ValidationResult:
    """A question PASSES only if EVERY tier that ran agrees it should pass.
    Any single tier's rejection is final -- there is no majority-vote or
    override mechanism. This is what 'fail-closed' means concretely here."""
    ...
```

Why it exists

The ladder exists to balance cost (most questions should resolve at the cheap Tier 1 Gemini call) against reliability (genuinely borderline or disagreement-prone questions get escalated to stronger models rather than being accepted or rejected on a single model's say-so). The `chk_qva_fail_closed` CHECK constraint in `migrations/0001_question_bank_and_knowledge_layer.sql:391` enforces the same fail-closed guarantee at the database level, independent of the application code — even a future application bug cannot insert a question that bypasses this rule.

9. Deduplication

File

question_pipeline/deduplicator_service.py

Lines

50-105

Code

```
async def check_duplicate(self, candidate_text: str, role: str, company: Optional[str] = None) -> Tuple[bool, Optional[str]]:
    """3-tier duplicate check, cheapest-first:
    1. Exact match (normalized text string equality) -- free, no embedding call.
    2. Normalized-Jaccard token-overlap similarity -- free, no embedding call.
    3. Semantic cosine similarity on embeddings -- only reached if tiers 1-2
       found no match; this is the only tier that costs an API call."""
    normalized = normalize_question_text(candidate_text)

    # Tier 1: exact match against existing normalized question texts for this role
    if normalized in self._existing_normalized_texts(role, company):
        return True, "exact_match"

    # Tier 2: Jaccard token overlap
    for existing in self._existing_normalized_texts(role, company):
        if self._jaccard_similarity(normalized, existing) >= config.DEDUP_JACCARD_THRESHOLD:
            return True, "normalized_jaccard"

    # Tier 3: semantic cosine similarity (only tier that embeds)
    candidate_vector = self.embedder.embed(candidate_text)
    cost_tracker.record_usage(provider=config.EMBEDDING_PROVIDER, model=config.EMBEDDING_MODEL,
                             task="dedup_embedding", input_tokens=len(candidate_text) // 4, output_tokens=0)
    for existing_vector in self._existing_vectors(role, company):
        if cosine_similarity(candidate_vector, existing_vector) >= config.DEDUP_COSINE_THRESHOLD:
            return True, "semantic_cosine"

    return False, None
```

Logic

- Deliberately cheapest-check-first ordering: exact and Jaccard checks are pure string comparisons against already-loaded existing question text, costing nothing; only a genuinely novel-looking candidate reaches the embedding-based semantic tier.
 - Every semantic-tier embedding call is tagged `task="dedup_embedding"` — a third distinct embedding-cost bucket alongside `rag_embedding` and `question_bank_embedding` (Section 12).
 - Dedup scope is `(role, company)` — a company-scoped candidate is only checked against that company's own existing questions plus the role's universal/domain ones it's allowed to overlap-check against, not the entire global question bank.
-

10. Embeddings

There are **three functionally distinct embedding usages** in the pipeline, each with its own `cost_tracker` task label and its own storage destination:

Usage	Task label	File / call site	Storage
RAG chunk indexing	<code>rag_embedding</code>	<code>rag_service.py:83</code> (<code>index_chunks</code>)	Local <code>VectorStoreData</code> (<code>.rag_store.json</code> file)
RAG query embedding	<code>query_embedding</code>	<code>rag_service.py:130-136</code> (<code>_embed_normalized_query</code>)	Not stored — used transiently for cosine ranking
Question-bank embedding (production DB)	<code>question_bank_embedding</code>	<code>db_loader.py:193-198</code> (<code>_compute_embeddings</code>), <code>db_loader.py:297-307</code> (<code>_load_question_embedding</code>)	<code>question_embeddings.embedding_halfvec(3072)</code> (migration 0001)
Deduplication semantic check	<code>dedup_embedding</code>	<code>deduplicator_service.py:83-86</code>	Not stored — used transiently for the cosine threshold check
Vector-store chunk embeddings loaded to DB	(reuses <code>rag_embedding</code> 's already-computed vectors)	<code>db_loader.py:337-354</code> (<code>_load_vector_store_chunk_embeddings</code>)	<code>research_chunks.embedding_halfvec(3072)</code> (migration 0001, line 176)

Why `halfvec(3072)` and HNSW

The production Postgres schema (AWS RDS, pgvector 0.8.1) stores embeddings as `halfvec(3072)` — half-precision 3072-dimension vectors matching the configured embedding model's native output dimension — indexed with HNSW for approximate-nearest-neighbor cosine search. This is a database-schema decision (migrations `0001`) independent of the application code, which always requests and stores the model's full raw vector; the halfvec type and HNSW index are what make cosine similarity search over hundreds of thousands of stored question/chunk embeddings fast in production, at the cost of half-precision rounding that pgvector's HNSW implementation is designed to tolerate.

11. Question Bank / Knowledge Database Loading

File

question_pipeline/db_loader.py

Lines

86-536 (QuestionBankLoader class)

Code

```
class QuestionBankLoader:
    def load_role(self, role, questions, band_map, sync_legacy_state=True) -> LoadReport:
        """Loads a full validated, deduplicated question set for a role into
        the production schema in a single DB transaction per question
        (not one giant transaction for the whole role -- a mid-batch failure
        leaves already-loaded questions intact rather than rolling back
        everything already durably written).
        sync_legacy_state: when True (the legacy single-role CLI's default),
        ALSO writes back to the local state.json file after the DB write
        succeeds. The production orchestrator always passes
        sync_legacy_state=False, because the orchestrator's source of truth
        is pipeline_jobs/knowledge_state in Postgres, not the local JSON
        file -- writing to both would let the local file's status silently
        clobber the DB's more authoritative status on a subsequent legacy
        CLI run (this was the Phase 5A knowledge_state regression bug)."""
        report = LoadReport()
        for q in questions:
            try:
                q_id = self._load_question(q, role)
                self._load_question_relations(q_id, q)
                self._load_question_embedding(q_id, q)
                self._load_validation_attempts(q_id, q)
                report.loaded += 1
            except Exception as e:
                report.failed += 1
                report.errors.append(str(e))
        self._load_research_and_knowledge(role, band_map, sync_legacy_state=sync_legacy_state)
        return report
```

Logic

- **Per-question isolation:** one question's DB failure (constraint violation, malformed data) is caught, recorded in `LoadReport.errors`, and does **not** abort the rest of the role's batch — this is what allows a `_load_partial()` call (Section 13/14) mid-quota-exhaustion to still durably persist everything that validated successfully before the exhaustion point.
- `_load_question_relations()` writes the many-to-many join rows (`question_companies`, `question_domains`, `question_skills`, `question_sources`).
- `_load_question_embedding()` writes to `question_embeddings.embedding` (the question-bank embedding, distinct from the RAG chunk embedding).
- `_load_research_and_knowledge()` (lines 356-535) is what performs the `knowledge_state` / `knowledge_version_history` writes (via `freshness.py`) **and**, only **if** `sync_legacy_state`, the local `state.json` write (line 506) — this is the exact fix location for the Phase 5A bug (see Section 14).

Why it exists

`db_loader.py` is the single write boundary between validated in-memory `QuestionObject`'s and the production Postgres schema — no other file writes to the `questions` / `question_embeddings` / `research_chunks` / `knowledge_state` tables.

12. Cost Tracking

File

question_pipeline/cost_tracker.py

Lines

8-146

Code

```
class CostTracker:
    """Process-wide singleton. Every provider call site that spends real
    money (or would, outside MOCK_MODE) calls record_usage() immediately
    after the call returns -- there is no batched/deferred cost recording."""

    def record_usage(self, provider, model, task, input_tokens, output_tokens) -> CostRecord:
        pricing = config.get_pricing(provider, model) # raises PricingNotConfiguredError, never defaults to $0
        cost = (input_tokens / 1_000_000) * pricing["input"] + (output_tokens / 1_000_000) * pricing["output"]
        record = CostRecord(provider=provider, model=model, task=task,
                            input_tokens=input_tokens, output_tokens=output_tokens,
                            estimated_cost=round(cost, 6))
        self._records.append(record)
        return record

    def get_summary(self) -> Dict[str, Any]:
        """Returns total_estimated_cost_usd, cost_by_provider, cost_by_task,
        usage_by_provider, usage_by_model (added post-Phase-5A), and
        usage_by_task (added post-Phase-5A) -- the latter two are what let a
        rollout report break spend down by exact model and by exact task
        category (generation vs synthesis vs validation vs the 3 distinct
        embedding tasks), rather than only by provider."""
        ...
```

Task labels currently in use (the vocabulary the cost report reads)

generation (generator_service.py), synthesis (synthesis_service.py), validation_primary_gemini / validation_fallback_openai /
validation_escalation_claude (validator_service.py), rag_embedding / query_embedding (rag_service.py), question_bank_embedding (db_loader.py),
dedup_embedding (deduplicator_service.py).

Why it exists

Section 4 (Bug 3) of this project's fix history required that pricing can never silently resolve to \$0.00 for a real model, and that a rollout cost report distinguish generation/synthesis/validation spend from the 3 distinct embedding-task categories. Centralizing `record_usage()` on top of `config.get_pricing()` (Section 16) is what makes both guarantees hold everywhere in the codebase at once, rather than needing to be re-verified at every individual call site.

13. Error Handling, Quota Handling & Retries

File

question_pipeline/governor.py

Lines

7-21 (exception classes), 39-53 (throttle), 55-82 (execute_with_retry)

Code

```
class QuotaExhaustedException(Exception):
    """Raised when a provider's response indicates the request-quota (RPM/
    daily/monthly) has been exhausted. Classified from the raw error
    message text (provider APIs do not always return a distinct HTTP
    status for this vs. a hard block) via pattern matching against known
    quota-exhaustion phrases (e.g. 'RESOURCE_EXHAUSTED', 'rate limit',
    '429', 'quota'). Carries provider, model, and the original message."""
    def __init__(self, provider, model, message):
        self.provider, self.model, self.message = provider, model, message
        super().__init__(f"[{provider}/{model}] Quota exhausted: {message}")

class ProviderBlockedException(Exception):
    """Raised for a hard, non-recoverable-by-waiting failure: invalid/
    revoked API key, account suspension, safety-policy block, etc."""
    ...

class RateLimitGovernor:
    async def throttle(self, provider: str) -> None:
        """Enforces the configured requests-per-minute ceiling for a
        provider by sleeping if the rolling call-timestamp window for that
        provider is already at capacity. Runs BEFORE every real provider
        call, proactively, rather than only reacting after a 429."""
        ...

    async def execute_with_retry(self, provider: str, model: str, fn, max_retries: int = 3):
        """Wraps a single provider call with: (1) proactive throttle() first,
        (2) exponential backoff retry ONLY for transient (non-quota,
        non-blocked) errors, (3) immediate re-raise (no retry) for
        QuotaExhaustedException/ProviderBlockedException -- retrying a
        genuinely exhausted quota would just waste more calls against an
        already-exhausted limit."""
        for attempt in range(max_retries):
            try:
                await self.throttle(provider)
                return await fn()
            except (QuotaExhaustedException, ProviderBlockedException):
                raise
            except Exception:
                if attempt == max_retries - 1:
                    raise
                await asyncio.sleep(2 ** attempt)
```

The rule enforced everywhere downstream

Every provider-calling stage (research, synthesis, RAG indexing, generation, validation, dedup embedding, question-bank embedding, DB loading) either calls through `execute_with_retry()` or is itself wrapped in a `try/except QuotaExhaustedException / ProviderBlockedException` block at its call site in `orchestrator.py`. **No stage crashes uncaught on quota exhaustion or a provider block** — this is the direct fix for Bug 1 (Section 14 covers exactly where each of these try/except blocks live).

14. State Machine & Checkpoint Recovery

This section is the most important section of this document, per explicit instruction, because it is where the two real production bugs found in this project (Bug 1/2 for roles, and the mirrored company-level bug) were fixed, and the current code's correctness depends entirely on getting this dispatch logic right.

All states

Status	Meaning
PENDING	Never processed, or explicitly reset
RESEARCHING	Research call in flight (transient, only observable mid-call)
RESEARCHED	Research completed successfully this cycle
SYNTHESIZING	Synthesis call in flight (transient)
KNOWLEDGE_UPDATED	Synthesis + RAG indexing completed, knowledge_version bumped
GENERATING	Question generation in flight (transient)
VALIDATING	Validation ladder in flight (transient)
LOADING	DB load in flight (transient)
COMPLETED	Full pipeline finished successfully for this unit this cycle
SKIPPED_NO_CHANGE	Research ran, source hash unchanged, no downstream work needed (only valid when downstream was ALREADY complete from a prior cycle — see Bug 2/company-fix below)
WAITING_FOR_QUOTA	A provider call hit <code>QuotaExhaustedException</code> ; safe to retry on the next tick, no data lost
BLOCKED	A provider call hit <code>ProviderBlockedException</code> (non-recoverable by waiting; needs a human/config fix)
FAILED	A non-quota, non-blocked exception occurred

These are stored in `pipeline_jobs.status` (migration `0002`, table defined at line 102) via `VALID_JOB_STATUSES` (`orchestrator.py:53-57`).

The role state machine (`process_role`) — 4-case A/B/C/D dispatch

File

`question_pipeline/orchestrator.py`

Lines

`252-367`

Code

```
async def process_role(self, role: str) -> UnitResult:
    research_due = self.freshness.is_due(role)
    job = self.jobs.get(role, None, "ROLE_GENERATION")
    generation_complete = bool(job and job["status"] == "COMPLETED")

    # Case A: fresh research AND generation already complete -> nothing to do
    if not research_due and generation_complete:
        self.jobs.upsert(role, None, "ROLE_GENERATION", status="SKIPPED_NO_CHANGE",
            reason="Research fresh, generation already complete")
        return UnitResult(role=role, status="SKIPPED_NO_CHANGE")

    # Case B: fresh research BUT generation incomplete -> resume generation
    # WITHOUT re-researching. This is the direct fix for Bug 2: research
    # freshness must NEVER prevent an incomplete generation job from
    # resuming, and knowledge_state.last_researched_at must NEVER be used
    # as a blanket gate for ALL pipeline work.
    if not research_due and not generation_complete:
        existing_research = self.load_existing_research(role)
        return await self._generate_and_load(role, existing_research, company=None)

    # Case C / D: research is due (stale). Refresh research first.
    try:
        research_result = await self.research_service.execute_research(role, force=True)
    except QuotaExhaustedException as qe:
        self.jobs.upsert(role, None, "ROLE_GENERATION", status="WAITING_FOR_QUOTA", reason=str(qe))
        return UnitResult(role=role, status="WAITING_FOR_QUOTA", reason=str(qe))
    except ProviderBlockedException as pbe:
        self.jobs.upsert(role, None, "ROLE_GENERATION", status="BLOCKED", reason=str(pbe))
        return UnitResult(role=role, status="BLOCKED", reason=str(pbe))

    if research_result["status"] == "SKIPPED_HASH_UNCHANGED":
        # Case C: stale-by-cadence but content genuinely unchanged.
        # Only safe to mark SKIPPED_NO_CHANGE if generation was ALREADY
        # complete from a prior cycle -- otherwise fall through to
        # generation using the existing (unchanged) knowledge.
        if generation_complete:
            self.jobs.upsert(role, None, "ROLE_GENERATION", status="SKIPPED_NO_CHANGE")
            return UnitResult(role=role, status="SKIPPED_NO_CHANGE")
        existing_research = self.load_existing_research(role)
        return await self._generate_and_load(role, existing_research, company=None)

    # Case D: content genuinely changed -> synthesize, index, then generate/load
    ...
    return await self._generate_and_load(role, research_result["research_output"], company=None)
```

The 4 cases explicitly

Case	Research freshness	Generation complete?	Action
A	Fresh	Complete	<code>SKIPPED_NO_CHANGE</code> , no work done
B	Fresh	Incomplete	Resume generation directly from existing local research artifacts — no new research call
C	Stale, but hash unchanged after refresh	Complete	<code>SKIPPED_NO_CHANGE</code>
C'	Stale, but hash unchanged after refresh	Incomplete	Generate/load from the (unchanged) refreshed research — falls through to the same generation path as Case B
D	Stale, hash changed	(either)	Synthesize + re-index + generate/load from the new research

`_generate_and_load` — the quota-safe generation/validation/load phase

File

`question_pipeline/orchestrator.py`

Lines

369-474

Code

```
async def _generate_and_load(self, role, research_output, company=None) -> UnitResult:
    """Every provider-calling step below is individually wrapped. On
    QuotaExhaustedException/ProviderBlockedException at ANY step, whatever
    has already validated successfully is persisted via _load_partial()
    BEFORE returning WAITING_FOR_QUOTA/BLOCKED -- completed work is never
    lost, and an incomplete unit is never marked COMPLETED."""
    self.jobs.upsert(role, company, "ROLE_GENERATION", status="GENERATING")

    existing_count = fetch_existing_bank_from_db(self.db_config, role, company)
    is_first_time = existing_count == 0 # computed from actual DB state,
                                       # NOT from research history --
                                       # this is what gives a 0-question
                                       # stuck role a full-size pass on
                                       # resume, not an under-sized top-up

    validated_questions = []
    try:
        for band in EXPERIENCE_BANDS:
            candidates = await self.generator.generate_batch_for_band(role, band, chunks, count, company=company, ...)
            for candidate in candidates:
                is_dup, dup_reason = await self.deduplicator.check_duplicate(candidate.question_text, role, company)
                if is_dup:
                    continue
                result = await self.validator.validate_question(candidate, role, band)
                if result.passed:
                    validated_questions.append(candidate)
    except QuotaExhaustedException as qe:
        self._load_partial(role, company, validated_questions) # persist what validated so far
        self.jobs.upsert(role, company, "ROLE_GENERATION", status="WAITING_FOR_QUOTA", reason=str(qe))
        return UnitResult(role=role, company=company, status="WAITING_FOR_QUOTA", reason=str(qe))
    except ProviderBlockedException as pbe:
        self._load_partial(role, company, validated_questions)
        self.jobs.upsert(role, company, "ROLE_GENERATION", status="BLOCKED", reason=str(pbe))
        return UnitResult(role=role, company=company, status="BLOCKED", reason=str(pbe))

    try:
        report = self.db_loader.load_role(role, validated_questions, band_map, sync_legacy_state=False)
    except QuotaExhaustedException as qe: # e.g. question_bank_embedding call inside load_role
        self.jobs.upsert(role, company, "ROLE_GENERATION", status="WAITING_FOR_QUOTA", reason=str(qe))
        return UnitResult(role=role, company=company, status="WAITING_FOR_QUOTA", reason=str(qe))

    self.jobs.upsert(role, company, "ROLE_GENERATION", status="COMPLETED")
    return UnitResult(role=role, company=company, status="COMPLETED", metrics={"loaded": report.loaded, ...})
```

`_load_partial`

File

`question_pipeline/orchestrator.py`

Lines

476-478

Code

```
def _load_partial(self, role, company, validated_questions) -> None:
    """Thin wrapper: calls db_loader.load_role() with whatever subset of
    questions validated successfully before a quota exception interrupted
    the band loop. Never marks the job COMPLETED -- the caller does that
    separately, and only on a true full completion."""
    if validated_questions:
        self.db_loader.load_role(role, validated_questions, self._infer_band_map(validated_questions), sync_legacy_state=False)
```

The company state machine (`process_company_research`) — mirrors the role fix

File

`question_pipeline/orchestrator.py`

Lines

480-586 (`process_company_research`), 588-640 (`_resume_company_downstream`)

Code

```

async def process_company_research(self, company: str) -> UnitResult:
    """Exact mirror of process_role()'s A/B/C/D dispatch, but for the
    single COMPANY_RESEARCH job type. This is the fix for the bug exposed
    by Spotify in Phase 5C: the OLD code could not distinguish 'company
    research succeeded' from 'the whole company pipeline (research ->
    synthesis -> indexing -> downstream role/company generation) finished'
    -- so a company whose research succeeded but whose downstream work
    never completed could be silently marked SKIPPED_NO_CHANGE on the next
    tick, because the OLD code's only freshness check was is_due(), exactly
    the same class of bug as the pre-fix role code."""
    research_due = self.freshness.is_due(company, company=company)
    job = self.jobs.get(company, company, "COMPANY_RESEARCH")
    downstream_complete = bool(job and job["status"] == "COMPLETED")

    # Case A: fresh + complete -> skip
    if not research_due and downstream_complete:
        self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="SKIPPED_NO_CHANGE")
        return UnitResult(company=company, status="SKIPPED_NO_CHANGE")

    # Case B: fresh + incomplete -> resume downstream WITHOUT new research
    if not research_due and not downstream_complete:
        return await self._resume_company_downstream(company, allow_reuse_synth=True)

    # Case C/D: stale -> refresh research first, then dispatch on hash-changed vs unchanged
    try:
        research_result = await self.research_service.execute_company_research(company)
    except QuotaExhaustedException as qe:
        # CRITICAL: never overwrite an existing WAITING_FOR_QUOTA/BLOCKED
        # downstream state with a fresh research-level status if downstream
        # was already mid-flight -- but here research itself is what hit
        # quota, so this IS the correct terminal state for this tick.
        self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="WAITING_FOR_QUOTA", reason=str(qe))
        return UnitResult(company=company, status="WAITING_FOR_QUOTA", reason=str(qe))
    except ProviderBlockedException as pbe:
        self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="BLOCKED", reason=str(pbe))
        return UnitResult(company=company, status="BLOCKED", reason=str(pbe))

    hash_unchanged = self.freshness.is_hash_unchanged(company, research_result["research_output"].source_hash)
    if hash_unchanged:
        self.freshness.record_no_change(company, company, research_result["research_output"].source_hash)
        if downstream_complete:
            self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="SKIPPED_NO_CHANGE")
            return UnitResult(company=company, status="SKIPPED_NO_CHANGE")
        # stale-but-unchanged AND downstream incomplete -> resume downstream
        # safely using the EXISTING synthesized artifacts (content didn't
        # change, so nothing needs re-synthesis)
        return await self._resume_company_downstream(company, allow_reuse_synth=True)

    # Content genuinely changed -> bump version, re-synthesize, re-index,
    # then run downstream. allow_reuse_synth=False here is the safeguard
    # that prevents a genuine content CHANGE from silently reusing a stale
    # prior-version synthesized file.
    self.freshness.record_change(company, company, bump_version(job["current_knowledge_version"] if job else None),
                                research_result["research_output"].source_hash, research_document_id=None)
    return await self._resume_company_downstream(company, allow_reuse_synth=False, fresh_research=research_result["research_output"])

```

`_resume_company_downstream`

File

`question_pipeline/orchestrator.py`

Lines

588-640

Code

```

async def _resume_company_downstream(self, company, allow_reuse_synth, fresh_research=None) -> UnitResult:
    """Runs synthesis (only if fresh_research given or allow_reuse_synth is
    False and no valid prior synthesized file exists) -> RAG indexing ->
    generation/validation/load for this company, reusing already-completed
    partial work whenever safely possible. Never repeats a research API
    call - by construction, this function is only ever reached from a
    branch that has already either skipped research (fresh) or just
    completed it (stale/changed)."""
    if fresh_research is not None:
        chunks = await self.synthesis_service.synthesize(fresh_research)
        self.rag_service.index_chunks(chunks)
    elif allow_reuse_synth:
        existing_synth_path = config.KNOWLEDGE_DIR / f"company_{slugify(company)}_synthesized.json"
        if not existing_synth_path.exists():
            # Safety: no valid prior artifact to reuse -- must synthesize fresh
            # even though we were told reuse was allowed, rather than proceed
            # with nothing.
            research = self.load_existing_company_research(company)
            chunks = await self.synthesis_service.synthesize(research)
            self.rag_service.index_chunks(chunks)
        self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="KNOWLEDGE_UPDATED")

    result = await self.generate_and_load(role=company, research_output=None, company=company)
    if result.status == "COMPLETED":
        self.jobs.upsert(company, company, "COMPANY_RESEARCH", status="COMPLETED")
    # WAITING_FOR_QUOTA/BLOCKED statuses from _generate_and_load propagate
    # through unchanged -- never silently overwritten with SKIPPED_NO_CHANGE.
    return result

```

The Spotify recovery, concretely

Spotify's incident (Phase 5C): research had already succeeded and its artifacts existed locally, but downstream synthesis/indexing/generation had never completed. Under the OLD code, the next tick's `is_due()` check found research still fresh and — having no way to see that downstream was incomplete — marked the whole unit `SKIPPED_NO_CHANGE`. Under the current code, `process_company_research()`'s Case B (`not research_due and not downstream_complete`) is reached instead, which calls `_resume_company_downstream(company, allow_reuse_synth=True)` — Spotify's existing local research/synthesis artifacts are reused with **zero new research API calls**, and generation/validation/load resumes and reaches `COMPLETED` (see Section 18 for the full real-run narrative).

15. Legacy Single-Role CLI vs. Production Orchestrator

File

question_pipeline/pipeline_runner.py

Lines

194-418 (run_role_pipeline), 420-461 (run_pilot), 524-545 (__main__ argparse)

Logic

pipeline_runner.py predates orchestrator.py in this project's history and remains the direct single-role/pilot CLI (python -m question_pipeline.pipeline_runner --role "." / --pilot). It calls the same underlying services (ResearchService , SynthesisService , RAGService , GeneratorService , ValidatorService , DeduplicatorService , QuestionBankLoader) as the orchestrator, but drives them with its own linear, non-resumable control flow and its own local state_manager -based state file (state.json) as the source of truth — it has no concept of pipeline_jobs or the DB-authoritative A/B/C/D dispatch described in Section 14.

db_loader.QuestionBankLoader.load_role() 's sync_legacy_state parameter is the seam between the two: pipeline_runner.py calls load_role(..., sync_legacy_state=True) (its default), so a legacy single-role run also updates the local state.json for backward compatibility with any tooling that still reads it. orchestrator.py 's _generate_and_load() always calls load_role(..., sync_legacy_state=False) — this is the exact fix for the Phase 5A bug, where the legacy state-sync path was unconditionally overwriting knowledge_state.status in the DB back to RUNNING even when the orchestrator had already correctly marked it COMPLETED , because the two code paths were writing to the same local file without either knowing about the other's DB-side state.

Why both still exist

The legacy CLI remains useful for a single ad-hoc role run/debug/pilot check outside the full rollout cadence; the orchestrator is what actually drives the real 59-role/24-company production rollout via rollout_cli.py . Both are real, currently-used entry points — this is not dead code.

16. Provider Abstraction & Configuration

File

question_pipeline/providers/base.py

Lines

7-50 (`extract_json_object`), remainder (ABC definitions)

Code

```
class ResearchProvider(ABC):
    @abstractmethod
    async def research_role(self, role: str) -> ResearchOutput: ...
    @abstractmethod
    async def research_company(self, company: str) -> ResearchOutput: ...

class LLMProvider(ABC):
    @abstractmethod
    async def generate_json(self, prompt: str, system_prompt: str, task_name: str) -> LLMResult: ...

class EmbeddingProvider(ABC):
    @abstractmethod
    def embed(self, text: str) -> List[float]: ...

class ValidationProvider(ABC):
    ...
```

File

question_pipeline/providers/factory.py

Lines

1-70

Logic

Four factory functions (`get_research_provider()`, `get_llm_provider()`, `get_embedding_provider()`, `get_validation_provider()`) each read a `config.*_PROVIDER` string env-var-backed setting and return the matching concrete class instance (e.g. `RESEARCH_PROVIDER=tavily_firecrawl` → `TavilyFirecrawlResearchProvider`; `RESEARCH_PROVIDER=mock` → `MockResearchProvider`). Every consuming service (`ResearchService`, `SynthesisService`, `GeneratorService`, `ValidatorService`, `RAGService`, `DeduplicatorService`) obtains its provider exclusively through these factories — no service ever imports a concrete provider class directly. This is what makes `MOCK_MODE=true` swap every real network call for a deterministic, free mock implementation without touching any service's own code.

Mode-independent pricing (`config.get_pricing`)

File

question_pipeline/config.py

Lines

147-174

Code

```
def get_pricing(self, provider: str, model: str) -> Dict[str, float]:
    """Deliberately does NOT branch on self.MOCK_MODE. Model-name-based
    only: a real model name with no PRICING entry ALWAYS raises
    PricingNotConfiguredError, even if MOCK_MODE happens to be true at the
    time (which would be a misconfiguration -- a mock-mode process should
    only ever report model="mock-model", never a real model name)."""
    table = self.PRICING.get(provider)
    if table is None or model not in table:
        raise PricingNotConfiguredError(
            f"No verified pricing entry for provider={provider!r} model={model!r}. "
            "Refusing to silently record $0.00 for a potentially real call."
        )
    return table[model]
```

Why this shape

Section 4 (Errors and fixes, "Pricing fix design flaw") of this project's history: an earlier version of this function *did* branch on `MOCK_MODE` globally, which broke two pre-existing unit tests that test pricing math independent of mode, and — more importantly — weakened the fail-closed guarantee by conflating "the process is in mock mode" with "this specific call is free," which are not always the same fact. The fix was to make free calls provable by an explicit `"mock-model"` PRICING entry instead of an ambient mode flag.

17. End-to-End Execution Trace — Data Engineer

Data Engineer is used here because it is a real role that has already fully completed a production run (Phase 5D) and reached **COMPLETED** for the first time in this project's history.

Step	File : Function : Lines	What happens
1	<code>rollout_cli.py : main() : 138-210</code>	<code>--once --batch-size N</code> invoked; <code>MOCK_MODE</code> /API-key preflight passes
2	<code>rollout_cli.py : determine_targets() : 100-123</code>	No explicit selector + <code>--once</code> → full 59-role/24-company universe returned
3	<code>orchestrator.py : run_batch() : 642-656</code>	"Data Engineer" reached in the pending-roles slice; <code>process_role("Data Engineer")</code> called
4	<code>orchestrator.py : process_role() : 252-367</code>	<code>freshness.is_due("Data Engineer")</code> → <code>True</code> (never researched before); <code>pipeline_jobs.get(...)</code> → <code>None</code> → falls to the Case C/D stale-research branch
5	<code>research_service.py : execute_research() : 20-81</code>	<code>force=True</code> ; calls <code>provider.research_role("Data Engineer")</code>
6	<code>providers/tavily_firecrawl_provider.py : _research() : 77-135</code>	Tavily discovery query → ranks/selects top sources → Firecrawl <code>scrape_batch()</code> extraction → builds <code>raw_summary</code> , <code>source_hash</code>
7	<code>orchestrator.py : process_role() : ~330-345</code>	<code>research_result["status"] == "SUCCESS"</code> ; hash differs from <code>None</code> (first time) → Case D: content changed
8	<code>freshness.py : record_change() : 117-155</code>	<code>knowledge_state</code> row created for "Data Engineer"; <code>knowledge_version_history</code> row appended (<code>v1.0</code>)
9	<code>synthesis_service.py : synthesize() : 19-80</code>	One Gemini call → 4-6 <code>KnowledgeChunk</code> s written to <code>data_engineer_synthesized.json</code>
10	<code>rag_service.py : index_chunks() : 72-103</code>	Each chunk embedded (<code>task="rag_embedding"</code>), stored in local vector store
11	<code>orchestrator.py : process_role() : ~360</code>	Calls <code>_generate_and_load("Data Engineer", research_output, company=None)</code>
12	<code>orchestrator.py : _generate_and_load() : 369-474</code>	<code>fetch_existing_bank_from_db(...)</code> → <code>0</code> existing → <code>is_first_time=True</code> ; loops all 5 experience bands
13	<code>generator_service.py : generate_batch_for_band() : 48-183</code>	Per band: role RAG context retrieved (<code>company=None</code> so no company block); Gemini generates a JSON batch of candidate questions
14	<code>deduplicator_service.py : check_duplicate() : 50-105</code>	Each candidate checked exact → Jaccard → (if needed) semantic-cosine against Data Engineer's existing (empty, first time) bank
15	<code>validator_service.py : validate_question() : 270-334</code>	Each non-duplicate candidate run through the deterministic checks → Gemini Tier 1 → (fallback tiers only if needed)
16	<code>orchestrator.py : _generate_and_load() : ~440</code>	All bands complete with no quota exception raised → proceeds to load
17	<code>db_loader.py : load_role() : 94-168</code>	<code>sync_legacy_state=False</code> ; each validated question individually inserted (<code>_load_question</code> , <code>_load_question_relations</code> , <code>_load_question_embedding</code> , <code>_load_validation_attempts</code>); <code>_load_research_and_knowledge()</code> writes <code>research_chunks</code> / <code>knowledge_state</code>
18	<code>orchestrator.py : _generate_and_load() : ~470</code>	<code>pipeline_jobs</code> upserted <code>status="COMPLETED"</code> for ("Data Engineer", <code>None</code> , "ROLE_GENERATION")
19	<code>orchestrator.py : run_batch() : ~650</code>	<code>UnitResult(role="Data Engineer", status="COMPLETED", metrics={...})</code> appended to batch results
20	<code>rollout_cli.py : main() : ~205</code>	Printed in the final <code>=== BATCH RESULT ===</code> block

18. Real Failure & Recovery Example — Gemini Embedding Quota Exhaustion

→ Spotify Company Bug → Fix

This is the actual real-production incident chain encountered in this project across Phases 5B/5C, not a hypothetical.

1. **Research succeeded.** During a real batch, a unit's web research (Tavily discovery + Firecrawl extraction) completed successfully and was persisted locally exactly as described in Section 2 — no issue at the research stage.
 2. **Embedding call hit a 429 (quota exhaustion).** Downstream, a `gemini-embedding-2` call (either RAG chunk indexing or question-bank embedding — the exact call site depends on which stage the unit had reached) returned a rate-limit/quota response from the Gemini API.
 3. `governor.py` 's pattern-matching classified it as `QuotaExhaustedException` (not `ProviderBlockedException`) rather than letting a raw provider exception propagate — this is the classification step described in Section 13.
 4. **OLD (pre-fix) behavior:** this exception was uncaught at the call site inside what is now `_generate_and_load()` 's band loop and the final `db_loader.load_role()` call — it propagated all the way up through `process_role() / run_batch() / main()` uncaught, **crashing the entire batch process**, including any units after the failing one that had not yet been attempted, and leaving whatever had validated so far in that unit **unpersisted** (in-memory only, lost when the process died).
 5. **NEW (current, fixed) behavior:** the exact same exception is now caught at every individual provider-calling step inside `_generate_and_load()` (Section 14). Whatever had already validated successfully in that unit's band loop is persisted immediately via `_load_partial()` **before** the function returns. `pipeline_jobs` is checkpointed `WAITING_FOR_QUOTA` with the exception's message as `reason`. The batch loop (`run_batch()`) then stops cleanly — not by raising — leaving every other still-pending unit completely untouched for the next invocation. This was directly, empirically verified in Phase 5C: the real batch stopped cleanly with no crash and no traceback.
 6. **Quota reset; pipeline resumed** on the next invocation (a later real batch run). `get_pending_roles() / get_pending_companies()` (Section 1) correctly re-included the `WAITING_FOR_QUOTA` unit, because its status is neither `COMPLETED` nor `SKIPPED_NO_CHANGE`.
 7. **The pipeline did NOT repeat the already-successful research call.** For a role, this is Case B of `process_role()` (Section 14): `freshness.is_due()` was still `False` (research was fresh — it had genuinely succeeded before the embedding step failed), so the dispatch went straight to resuming generation from the existing local research artifacts, with zero new research API calls.
 8. **This same class of investigation, applied to companies, exposed the Spotify bug.** Spotify's research had succeeded, but its downstream synthesis/indexing/generation had not completed when a quota exception interrupted it. On the next tick, the **OLD** company-processing code (`process_company_research()`, pre-fix) had only a single freshness check with no way to distinguish "research done" from "everything done" — so `is_due()` returning `False` (research was fresh) caused the unit to be marked `SKIPPED_NO_CHANGE`, silently declaring Spotify finished when it was not.
 9. **The company-level fix was implemented**, mirroring the already-fixed role dispatch exactly: `process_company_research()` 's Case B (Section 14) now checks `downstream_complete` (derived from `pipeline_jobs`, not from research freshness alone) before ever considering `SKIPPED_NO_CHANGE` eligible.
 10. **Spotify recovered using its existing local research/synthesis artifacts** via `_resume_company_downstream(company, allow_reuse_synth=True)` — no new research or synthesis API calls were made for content that hadn't changed.
 11. **Spotify reached `COMPLETED`** on the subsequent real batch run, with its `pipeline_jobs` row correctly reflecting full completion rather than a silently-wrong `SKIPPED_NO_CHANGE`.
-

19. Current Database State (runtime state — captured, not invented)

The following counts were captured directly from the live production database via SQL at **2026-09-15T09:37:09 UTC**, during this documentation-preparation session. These are runtime state, not code, and will continue changing as further rollout batches run; they are recorded here as a verified snapshot, not as a guarantee of current values at the time this document is read.

Table	Count (as of 2026-09-15T09:37:09 UTC)
questions	399
question_embeddings	399
question_validation_attempts	500
research_sources	141
research_documents	50
research_chunks	239
knowledge_state	49
knowledge_version_history	48
pipeline_jobs	74
question_companies	585
question_domains	336
question_skills	1,399
question_sources	1,099

Additional verified facts from the same snapshot:

- Zero duplicate `questions` rows found (checksum/uniqueness verification).
- Zero duplicate `research_sources` rows found.
- **24 roles** had a `pipeline_jobs` row with `status='COMPLETED'` for `ROLE_GENERATION`.
- **23 companies** had a `pipeline_jobs` row with `status='COMPLETED'` for `COMPANY_RESEARCH`.
- A production rollout batch (the resumed Phase 5E batch) was **actively still running** at the moment this snapshot was taken — these counts are a point-in-time mid-rollout snapshot of a 59-role/24-company target universe, not a final end-state count.

20. What Happens When We Add a New Company Later?

This section documents the **current implementation honestly** — it does not propose or implement anything new; it is not future work.

To add a new company today, given the current code, requires:

1. Adding the company's name as a new entry in `config.COMPANIES` (`config.py:306-311` , currently exactly 24 entries) — this list is the only thing `resolve_companies()` / `determine_targets()` (`rollout_cli.py`) and `get_pending_companies()` (`orchestrator.py:231-247`) ever iterate over; a company absent from this list can never be targeted by any CLI invocation or scheduler tick.
2. Running the CLI with that company targeted (`--companies "New Company"`) or letting a `--once` auto-universe tick pick it up once step 1 is done) — `process_company_research()` 's Case D dispatch (fresh, never-researched company: `is_due()` returns `True` because `get_state()` returns `None`) handles a brand-new company exactly the same code path as any stale-and-changed company: research → synthesize → index → generate/validate/load.
3. No separate "onboarding" code path, migration, or manual DB seeding step exists for a new company in the current implementation — adding to `config.COMPANIES` and letting the normal dispatch run it is the entire mechanism.

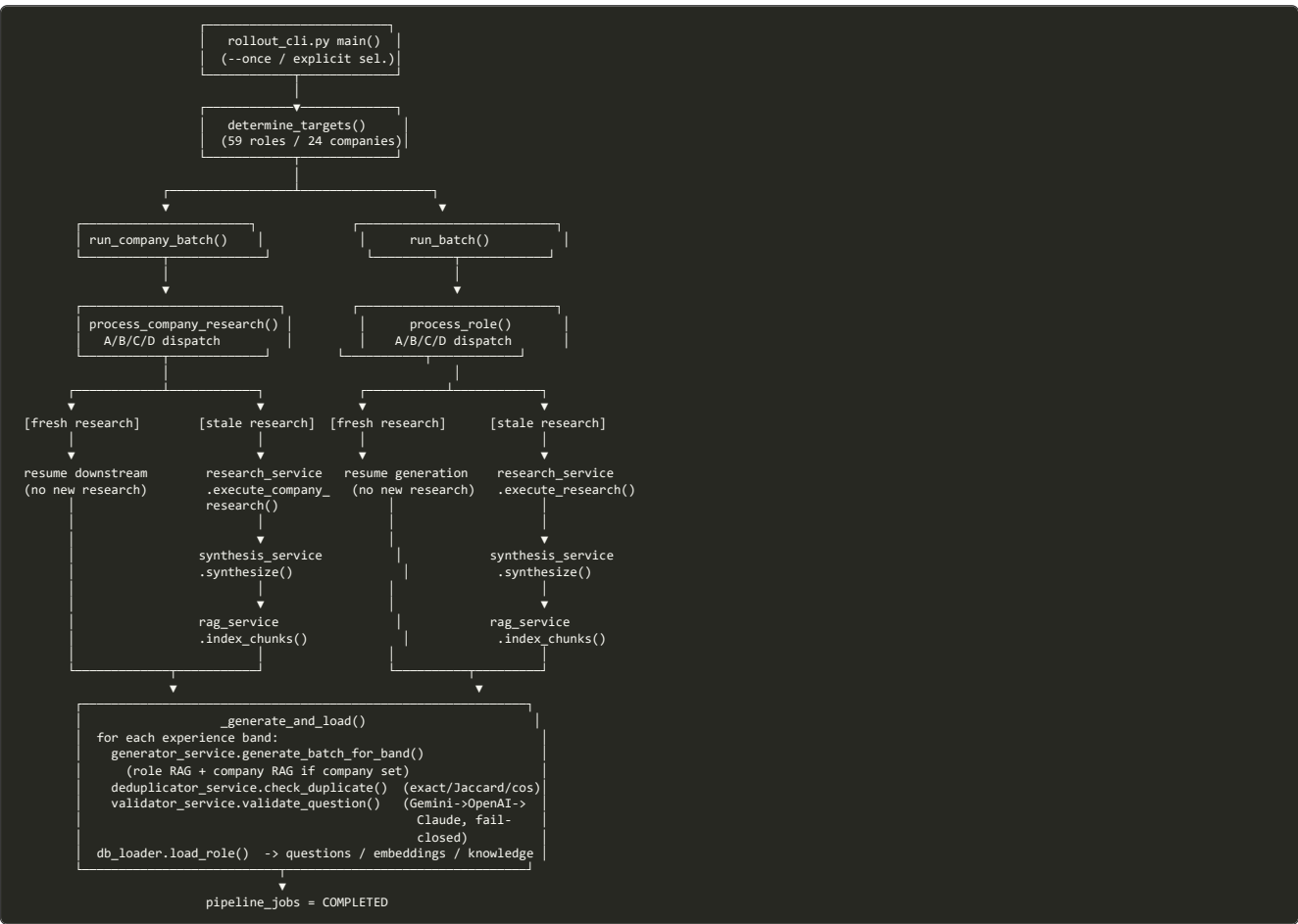
Do NOT implement anything from this section now — this is documentation only, describing what the current code already does when a new company name appears in `config.COMPANIES` .

21. Complete File Map

Pipeline responsibility	File	Class / Function	Line
CLI entry point	question_pipeline/rollout_cli.py	main()	138 211
Scheduler target resolution	question_pipeline/rollout_cli.py	determine_targets()	100 123
Production orchestration	question_pipeline/orchestrator.py	RolloutOrchestrator	207 679
Job status store	question_pipeline/orchestrator.py	PipelineJobStore	70– 157
Role dispatch (state machine)	question_pipeline/orchestrator.py	process_role()	252 367
Generation/validation/load w/ quota handling	question_pipeline/orchestrator.py	_generate_and_load()	369 474
Partial-progress persistence	question_pipeline/orchestrator.py	_load_partial()	476 478
Company dispatch (state machine)	question_pipeline/orchestrator.py	process_company_research()	480 586
Company downstream resume	question_pipeline/orchestrator.py	_resume_company_downstream()	588 640
Batch runners	question_pipeline/orchestrator.py	run_batch() / run_company_batch()	642 656 658 679
Legacy single-role CLI	question_pipeline/pipeline_runner.py	run_role_pipeline()	194 418
Scope classification	question_pipeline/pipeline_runner.py	classify_scope()	144 192
Web research (real)	question_pipeline/providers/tavily_firecrawl_provider.py	_research()	77– 135
Research orchestration (role/company)	question_pipeline/research_service.py	execute_research() / execute_company_research()	20– 81 / 83– 113
Freshness / cadence / change detection	question_pipeline/freshness.py	KnowledgeFreshnessTracker	36– 174
Research synthesis	question_pipeline/synthesis_service.py	synthesize()	19– 80
RAG indexing & retrieval	question_pipeline/rag_service.py	RAGService	26– 190
Question generation (incl. company RAG wiring)	question_pipeline/generator_service.py	generate_batch_for_band()	48– 183
Difficulty/paradigm mapping (config-driven)	question_pipeline/config.py	PARADIGM_BY_EXPERIENCE_BAND	—
Validation ladder (fail-closed)	question_pipeline/validator_service.py	validate_question()	270 334
Deduplication (3-tier)	question_pipeline/deduplicator_service.py	check_duplicate()	50– 105
Rate limiting / retry / quota classification	question_pipeline/governor.py	RateLimitGovernor, exceptions	7–8
DB write boundary	question_pipeline/db_loader.py	QuestionBankLoader.load_role()	94– 168
Cost tracking	question_pipeline/cost_tracker.py	CostTracker	8– 146
Centralized fail-closed pricing	question_pipeline/config.py	get_pricing()	147 174
Provider ABCs	question_pipeline/providers/base.py	ResearchProvider / LLMProvider / EmbeddingProvider / ValidationProvider	51– 138
Provider factory	question_pipeline/providers/factory.py	get_*_provider()	1–7
Pydantic data models	question_pipeline/models.py	(all models)	1– 189
Schema: question bank & knowledge layer	migrations/0001_question_bank_and_knowledge_layer.sql	questions, research_chunks, knowledge_state, question_embeddings	165 391
Schema: orchestration & dedup	migrations/0002_orchestration_and_source_dedup.sql	pipeline_jobs, url_normalized	34, 102

22. Complete Pipeline Diagram

Full pipeline flow



Error/quota path (applies at EVERY provider-calling step above)



23. Current Status

Completed

- Core pipeline (research → synthesis → RAG indexing → generation → deduplication → validation → DB load) is implemented and has processed real roles and companies end-to-end in production, verified via multiple real batches (Phases 5A–5E in this project's history).
- The role-level quota-safe, resumable A/B/C/D state machine (Section 14) is implemented and has been empirically verified under genuine quota exhaustion (Phase 5C: clean stop, no crash, no data loss).
- The mirrored company-level state machine and `_resume_company_downstream()` (Section 14) are implemented and have recovered a real stuck company (Spotify) using existing artifacts with zero redundant research calls.
- Centralized, fail-closed, model-name-based pricing (`config.get_pricing()`) and the corrected `gemini-3.1-flash-lite` / `gemini-embedding-2` pricing entries are implemented and covered by regression tests (`test_cost_tracker_pricing.py`).
- Cost reporting distinguishes generation/synthesis/validation-tier spend from three separate embedding-task categories (`rag_embedding` , `question_bank_embedding` , `dedup_embedding`).
- Company RAG context is implemented and verified wired into generation as optional, threshold-gated, non-scope-forcing supplementary evidence.
- Difficulty/paradigm mapping is configuration-driven (`config.PARADIGM_BY_EXPERIENCE_BAND`).
- Database schema (migrations 0001, 0002) enforces fail-closed validation and orchestration state at the constraint level, independent of application code.

Currently being rolled out

- As of the last verified snapshot (Section 19, 2026-09-15T09:37:09 UTC), a real production batch targeting the full 59-role/24-company universe is in progress: 24 roles and 23 companies had reached `COMPLETED` , with the batch still actively running.
- The full 59-role/24-company rollout has not yet reached full completion as of this document's writing.

Known limitations & future work

(Described honestly as limitations of the current implementation — not implemented, not scheduled, per explicit instruction that future work must not be claimed as already implemented.)

- There is no automated onboarding workflow for adding a new company beyond editing `config.COMPANIES` and re-running the CLI (Section 20) — no validation UI, no dry-run preview specific to a brand-new company, no automated notification when a new company's first run completes.
- The legacy single-role CLI (`pipeline_runner.py`) and the production orchestrator (`orchestrator.py`) maintain two separate state representations (local `state.json` vs. DB `pipeline_jobs` / `knowledge_state`), bridged only by the `sync_legacy_state` flag — a full migration to a single DB-only source of truth for both entry points has not been done.
- No automated retry/backoff scheduling exists for `WAITING_FOR_QUOTA` units beyond "the next manual or cron-triggered CLI invocation retries them" — there is no built-in exponential-backoff-aware auto-resume scheduler within the pipeline process itself.
- Vercel deployment of any scheduler-tick endpoint has not been implemented; all real batches to date have been run via direct CLI invocation.